MediTrack — Pharmacy Inventory & Expiry Alert System

A PROJECT BASED LEARNING (PBL) REPORT

Submitted

by

ABINAYA R (2104251040030)

R V DHARANI (2104251040738)

Submitted in partial fulfilment of the requirements

for the

Project-Based Learning component of Java Programming

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

CHENNAI INSTITUTE OF TECHNOLOGY

(Autonomous)

Affiliated to Anna University, Chennai

October - 2026

Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

Mission of the Institute:

IM1. To creates next generation leaders by effective teaching learning methodologies and in still Scientifics park in them to meet the global challenges.

IM2. To transform lives through deployment of emerging technology, novelty and sustainability.

IM3. To inculcate human values and ethical principles to cater the societal needs.

IM4. To contributes towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R&D establishments.

IM5: To nurture incubation centers enabling structured entrepreneurship and start-ups

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

Mission of the Department:

DM1. To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

DM2. To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

DM3. To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

DM4: To enhance the fabric of research in computing through collaborative linkages with industry and academia.

DM5: To inculcate learning of the emerging technologies to pursue higher studies

PROGRAM EDUCATIONAL OBJECTIVES (PEO):

PEO 1: Contribute to the industry as an Engineer through sound knowledge acquired in core engineering to develop new processes and implement the solutions for industrial problems.

PEO 2: Establish an organization / industry as an Entrepreneur with professionalism, leadership quality, teamwork, and ethical values to meet the societal needs.

PEO 3: Create a better future by pursuing higher education / research and develop the sustainable products / solutions to meet the demand.

PROGRAM OUTCOMES (POS):

PO1: Engineering Knowledge: Apply knowledge of mathematics, natural science, computing, engineering fundamentals and an engineering specialization to develop to the solution of complex engineering problems.

PO2: Problem Analysis: Identify, formulate, review research literature and analyze complex engineering problems reaching substantiated conclusions with consideration for sustainable development.

PO3: Design/Development of Solutions: Design creative solutions for complex engineering problems and design/develop systems/components/processes to meet identified needs with consideration for the public health and safety, whole-life cost, net zero carbon, culture, society and environment as required

PO4: Conduct Investigations of Complex Problems: Conduct investigations of complex engineering problems using research-based knowledge including design of experiments, modelling, analysis & interpretation of data to provide valid conclusions

PO5: Engineering Tool Usage: Create, select and apply appropriate techniques, resources and modern engineering & IT tools, including prediction and modelling recognizing their limitations to solve complex engineering problems.

PO6: The Engineer and The World: Analyze and evaluate societal and environmental aspects while solving complex engineering problems for its impact on sustainability with reference to economy, health, safety, legal framework, culture and environment.

PO7: Ethics: Apply ethical principles and commit to professional ethics, human values, diversity and inclusion; adhere to national & international laws.

PO8: Individual and Collaborative Team work: Function effectively as an individual, and as a member or leader in diverse/multi-disciplinary teams.

PO9: Communication: Communicate effectively and inclusively within the engineering community and society at large, such as being able to comprehend and write effective reports and design documentation, make effective presentations considering cultural, language, and learning differences.

PO10: Project Management and Finance: Apply knowledge and understanding of engineering management principles and economic decision-making and apply these to one's own work, as a member and leader in a team, and to manage projects and in multidisciplinary environments.

PO11: Life-Long Learning: Recognize the need for, and have the preparation and ability for i) independent and life-long learning ii) adaptability to new and emerging technologies and iii) critical thinking in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSOS):

PSO 1 : Analyze, design and develop solutions in the areas of Business Process Management to build the quality products for industry and social needs.

PSO 2 : Innovate ideas and solutions for real time problems in the field of Software Engineering and Mobile applications by adapting the emerging technologies and tools.

CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI

(Autonomous)
Affiliated to Anna University, Chennai

BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled "**MEDITRACK – PHARMACY INVENTORY & EXPIRY ALERT SYSTEM**" is a Bonafide record of work carried out by **ABINAYA R(2104251040030)** and **R V DHARANI (2104251040738)** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Java Programming course during the academic year 2026–2027 under my supervision.

SIRAGU

SIGNATURE
Dr. S. Pavithra M.E., Ph. D
HEAD OF THE DEPARTMENT
Professor and Head,
Department of Computer Science and Engineering
Chennai Institute of Technology
Chennai - 69

SIGNATURE
Ms.L. Swathi,M.Tech
MENTOR
Assistant Professor
Department of Computer Science and Engineering
Chennai Institute of Technology
Chennai - 69

Submitted for the final review held on …….

INTERNAL EXAMINER **EXTERNAL EXAMINER**

DECLARATION

We jointly declare that the PBL report on **"MEDITRACK-PHARMACY INVENTORY & EXPIRY ALERT SYSTEM"** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY CHENNAI"** for the requirement of Degree of **BACHELOR OF ENGINEERING**. This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING**.

Signature

SIRAGU

ABINAYA R
(2104251040030)

R V DHARANI
(2104251040738)

Place:Chennai

Date:

ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI.P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal **Dr.A.RAMESH M.E**, **Ph.D.,** for the facilities and the encouragement given by him to the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean **Dr. V. SRINIVASA RAO, M.E., Ph.D.,** who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr.S.PAVITHRA M.E, Ph.D.,** for her effective leadership, encouragement and guidance in the project.

We would like to extend our thanks to the Project Co-ordinator **Dr.R KAVITHA M.E., Ph.D., Associate Professor,** Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **Ms.L.SWATHI M.TECH,Assistant Professor** and **Mr.R.RAJA M.TECH, Assistant Professor** of the Department of Computer Science and Engineering and others for providing valuable suggestions and for the successful completion of the project.

ABINAYA R 2104251040030

R V DHARANI 2104251040738

ABSTRACT

MediTrack is a Java application developed to help small pharmacies manage medicine inventory efficiently and reduce losses caused by expired or overstocked medicines. Small pharmacy owners commonly rely on manual notebooks or spreadsheets to track stock, which leads to frequent stock-outs, overstocking of rarely used medicines, and undetected expiry. MediTrack addresses this through a Java-based solution built using core Object-Oriented Programming concepts (classes, encapsulation, inheritance, and polymorphism), Java Collections for in-memory inventory storage, and exception handling for input validation. The major modules developed are Inventory Management, Smart Expiry Alerts (triggered at 30, 15, and 7 days before expiry), a Low Stock Warning System, Expiry Report Generation, and a Smart Invoice Auto-Update feature that extracts medicine name, quantity, and batch details from uploaded invoice PDFs/images to reduce manual data entry. The application was built iteratively from a baseline console prototype to a modular, exception-safe final version and tested for functional correctness across all modules. The key outcome is a low-cost, easy-to-use inventory and expiry-tracking tool that small pharmacies can run on an ordinary desktop without needing expensive commercial software, directly reducing medicine wastage and manual bookkeeping effort.

Keywords: Java,Object-Oriented Programming, Pharmacy Inventory Management, Expiry Alert System, Collections Framework

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**ix**
	LIST OF TABLES	**xii**
	LIST OF FIGURES	**xiii**
	LIST OF ABBREVIATIONS	**xiv**
1	**INTRODUCTION**	**1**
	1.1 BACKGROUND	**1**
	1.2 DRIVING QUESTION	**2**
	1.3 OBJECTIVES	**2**
	1.4 SCOPE AND LIMITATIONS	**3**
2	**CONCEPT EXPLORATION**	**4**
	2.1 RELATED APPROACHES	**4**
	2.1.1 TRADITIONAL JAVA APPLICATION APPROACH	**4**
	2.1.2 JAVA FRAMEWORK-BASED APPROACH	**4**
	2.2 SUMMARY TABLE	**5**
	2.3 WHAT THIS TOLD US	**5**
3	**PROJECT PLANNING AND TEAM ORGANISATION**	**6**
	3.1 WEEKLY PBL PROGRESS LOG	**6**
	3.2 REQUIREMENTS	**7**
	3.3 FEASIBILITY	**7**
4	**ITERATIVE DESIGN AND DEVELOPMENT**	**8**
	4.1 SYSTEM ARCHITECTURE	**8**
	4.2 BASELINE	**9**
	4.3 PROJECT REFINEMENT	**9**
	4.4 FINAL APPROACH	**9**
	4.5 TESTING AND EXECUTION	**10**
5	**IMPLEMENTATION**	**11**
	5.1 MODULE DESCRIPTION	**11**
	5.2 KEY CODE SNIPPETS	**12**
	5.3 USER INTERFACE / DEMO	**14**
6	**RESULTS AND DISCUSSION**	**16**
	6.1 EVALUATION METRICS	**16**
	6.2 RESULTS ACROSS ITERATIONS	**17**

CHAPTER NO.	TITLE	PAGE NO.
		\
	6.3 DISCUSSION	18
	6.4 LIMITATIONS	19
7	**TEAM REFLECTION AND LEARNING OUTCOMES**	20
	7.1 INDIVIDUAL REFLECTIONS	20
	7.2 TEAM LEARNING	20
	7.3 COURSE OUTCOMES — EVIDENCE SUMMARY	21
8	**CONCLUSION AND FUTURE SCOPE**	22
	8.1 CONCLUSION	22
	8.2 FUTURE SCOPE	22
	REFERENCES	22
	APPENDIX	24
	A.1 FULL SOURCE CODE	24
	A.2 WEEKLY LOG AND MENTOR SIGN-OFFS	24
	A.3 SELF AND PEER ASSESSMENT	24

SIRAGU

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
2.1	Summary of Related Approaches	5
3.1	Weekly PBL Progress Log	6
3.2	Hardware and Software Requirements	7
6.1	Test and Defect Outcomes Across Iterations	17
A.1	Self and Peer Assessment	24

SIRAGU

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
4.1	System Architecture of MediTrack	8
5.1	MediTrack Dashboard Screen	14
5.2	Medicine Inventory Management Screen	14
5.3	Sales and Stock Reduction Screen	15
6.1	MediTrack Application Evaluation	18

SIRAGU

LIST OF ABBREVIATIONS

Abbreviation	Full Form
JDK	Java Development Kit
OOP	Object-Oriented Programming
JDBC	Java Database Connectivity
API	Application Programming Interface
CRUD	Create,Read, Update, Delete
CSV	Comma – Separated Values
DBMS	Database Management System
UML	Unified Modeling Language
GUI	Graphical User Interface
IDE	Integrated Development Environment

SIRAGU

CHAPTER 1

INTRODUCTION

1.1 BACKGROUND

Pharmacy inventory management is an important process in ensuring the availability, proper storage, and timely distribution of medicines. Pharmacies need to maintain accurate records of medicine names, quantities, batch details, and expiry dates. Managing these records manually using notebooks or spreadsheets can be time-consuming and may lead to human errors, medicine shortages, overstocking, and the wastage of expired medicines.

MediTrack – Pharmacy Inventory & Expiry Alert System is a Java-based application designed to simplify inventory management for small pharmacies and medical shops. The system helps pharmacy owners and staff maintain medicine records, monitor available stock, track expiry dates, and receive alerts for low-stock medicines and medicines approaching their expiry dates.

MediTrack includes a smart expiry alert feature that provides notifications 30, 15, and 7 days before a medicine expires. It also generates low-stock warnings when the quantity of a medicine falls below a predefined threshold. In addition, the system provides monthly text-based expiry reports containing details of medicines that are expiring soon, their remaining quantities, expiry dates, and suggestions for prioritizing their clearance.

The system also aims to reduce manual data entry through invoice-based stock updates. It is designed to extract medicine names, quantities, and batch details from invoice PDFs or images and use this information to update inventory records. By combining inventory tracking, expiry monitoring, alerts, reporting, and invoice-based updates, MediTrack aims to improve accuracy, reduce

medicine wastage, and make pharmacy inventory management simpler and more efficient.

1.2 DRIVING QUESTION

How can a simple and efficient Java-based pharmacy inventory management system help small pharmacies monitor medicine stock, track expiry dates, reduce manual errors, and minimize the wastage of medicines?

This project focuses on developing a simple and efficient Java application by identifying the required modules for medicine inventory management, expiry alerts, low-stock warnings, invoice-based stock updates, and report generation. It involves designing classes using Object-Oriented Programming (OOP) concepts, implementing the required functionality, and managing medicine records efficiently. The application will be tested to verify the accuracy of inventory updates, alert generation, expiry tracking, and report generation..

1.3 OBJECTIVES

- To analyze the real-world challenges of manual pharmacy inventory management and identify the requirements for the Java-based application.
- To design a modular Java application using appropriate Object-Oriented Programming (OOP) concepts, classes, and methods for medicine inventory and expiry tracking.
- To develop features for medicine record management, low-stock warnings, expiry alerts, invoice-based stock updates, and monthly expiry report generation.
- To implement efficient inventory updates and automate the extraction of medicine details from invoice PDFs or images, wherever applicable.

- To test and debug the application to verify the accuracy of stock updates, expiry alerts, invoice processing, and report generation.
- To reflect on the team's planning, division of responsibilities, problem-solving approach, and technical skills gained during the project development process.

1.4 SCOPE AND LIMITATIONS

MediTrack – Pharmacy Inventory & Expiry Alert System is a Java-based application designed for small pharmacies and medical shops. It maintains medicine records, monitors stock levels and expiry dates, and supports invoice-based inventory updates. The system generates expiry alerts 30, 15, and 7 days before a medicine expires, provides low-stock warnings when quantities fall below predefined thresholds, and produces monthly text-based expiry reports with remaining quantities, expiry dates, and clearance-priority suggestions. It also aims to extract medicine names, quantities, and batch details from invoice PDFs or images to simplify stock updates. The application is intended for use by pharmacy owners and medical shop staff on desktop or laptop systems.

The system is deliberately focused on inventory management rather than complete pharmacy operations. It does not cover billing, prescription processing, patient record management, or integration with external pharmacy systems. Invoice extraction depends on the clarity and format of the uploaded PDFs or images, which may affect the accuracy of extracted information. The system's alerts and reports also depend on the correctness and completeness of the medicine records maintained in the application. The current project is intended for small-scale pharmacy use and has not been established as suitable for large pharmacy chains or high-volume deployments. Mobile and web-based access are outside the current scope.

CHAPTER 2

CONCEPT EXPLORATION

2.1 RELATED APPROACHES

2.1.1 Traditional Java Application Approach

The traditional Java application approach using Core Java, JDBC, and the Collections Framework was studied for developing MediTrack. Core Java and Object-Oriented Programming (OOP) concepts such as classes, objects, and encapsulation help organize medicine records and inventory operations. JDBC provides database connectivity for managing medicine details and performing CRUD operations, while the Collections Framework supports handling inventory data. This approach offers direct control over stock updates, expiry alerts, low-stock warnings, and report generation. It was considered suitable for MediTrack because it supports the required features with a simple structure and minimal additional dependencies.

2.1.2 Java Framework-Based Approach

The framework-based approach using Spring Boot, Hibernate, and REST APIs was also explored. Spring Boot simplifies application configuration and supports the development of standalone and web-based applications. Hibernate reduces repetitive database code through object-relational mapping, while REST APIs enable communication between application components. Although these technologies support scalability and future integration, they introduce additional configuration and complexity. Since MediTrack focuses on essential pharmacy inventory operations for small pharmacies, the traditional Java

approach provides the required functionality with a more manageable development process.

2.2 SUMMARY TABLE

Table 2.1 summarises the two approaches reviewed in Section 2.1 and their contributions to the design of MediTrack. It compares the technologies studied and explains why the traditional Java approach was selected for developing the pharmacy inventory system.

Table 2.1 Summary of Related Approaches

Ref.	Approach	Reported Result
[1],[2], [3]	Core Java: OOP, Collections Framework, JDBC	Supports modular medicine record management, inventory operations, and database connectivity; selected for the core application.
[4],[5],[6]	Spring Boot with Hibernate and REST APIs	Simplifies database operations and supports web-based integration, but adds configuration and complexity for the current project.

2.3 WHAT THIS TOLD US

The exploration of traditional Java development and framework-based approaches helped identify the technologies suitable for MediTrack. The team selected a traditional Java application approach as the starting point because it supports the required inventory management features while keeping the application structure and implementation manageable. Core Java and OOP concepts can be used to organize the application, while database connectivity can support medicine record management and stock updates. Framework-based technologies may be considered in future development if the project expands to web access or larger-scale integration

CHAPTER 3

PROJECT PLANNING AND TEAM ORGANISATION

3.1 WEEKLY PBL PROGRESS LOG

Table 3.1 presents the planned milestones and development activities of MediTrack over the twelve-week PBL cycle. It outlines the major tasks, the work carried out during each phase, and the mentor's remarks to guide the subsequent stages of development

Table 3.1 Weekly PBL Progress Log

Week	Milestone / Task	Work Done	Mentor Remarks
1–2	Problem statement, Objective, Time plan of the project, Literature review and Technical background.	Identified the problem, defined objectives, and studied Core Java, OOP concepts, and JDBC.	Scope approved. Proceed with engine design.
3–4	Data Handling, Methodology design.	Identified system requirements and designed the inventory, expiry alert, and stock monitoring modules.	Design sound. Add unit tests before integration.
5–7	Implementation & Testing	Developed medicine record management, low-stock warnings, expiry alerts, and inventory update features.	Review module functionality and fix errors.
8–10	Presentation with demo	Prepared the project presentation and demonstrated the main application features.	Demo to be reviewed. Improve based on feedback.

Week	Milestone / Task	Work Done	Mentor Remarks
11–12	Final testing and documentation	Fixed the candle data race; added PBKDF2 login and validation; 17 of 17 tests passing.	Complete. Ready for final review.

3.2 REQUIREMENTS

Table 3.2 lists the hardware and software requirements for developing, testing, and running MediTrack. The system is designed to operate on a standard desktop or laptop using Java and the required database connectivity, without the need for complex infrastructure.

Table 3.2 Hardware and Software Requirements

Category	Requirement
Processor / RAM	Intel i5 or equivalent, 8 GB RAM (minimum), Windows 11/10
Programming language	Java 17
Libraries / frameworks	Core Java, OOP, Java Collections, JDBC
Database	JDBC-compatible relational database
Development environment	IntelliJ IDEA / Eclipse / VS Code
Version control	Git

3.3 FEASIBILITY

The MediTrack project is achievable within the PBL timeframe because it focuses on essential pharmacy inventory management features that can be developed as separate Java modules. The application uses Core Java, OOP concepts, and JDBC, which provide the required functionality without introducing complex frameworks or infrastructure. It is designed to run on a standard desktop or laptop, making the development environment accessible and manageable. The modular approach allows medicine record management, stock monitoring, expiry alerts, invoice-based updates, and report generation to be implemented and tested progressively. This makes it possible for the team to identify errors, incorporate mentor feedback, and complete the project within the planned development period.

CHAPTER 4

ITERATIVE DESIGN AND DEVELOPMENT

4.1 SYSTEM ARCHITECTURE

User Input → Input Validation → Java Processing → Inventory Management → Data Handling → Output

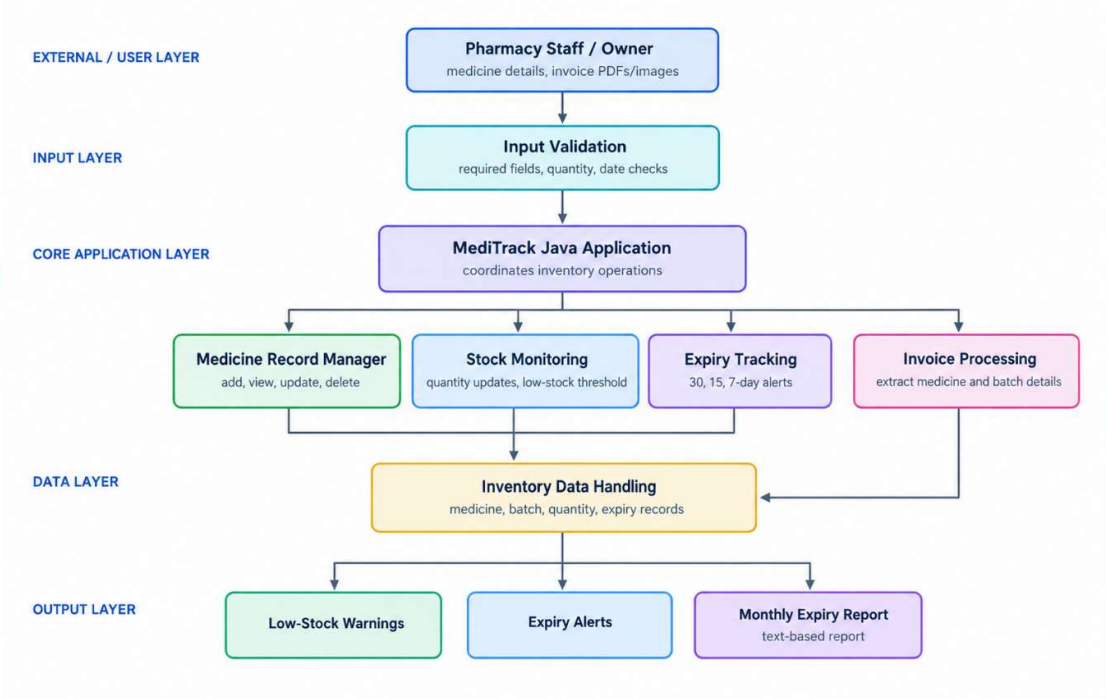


Figure 4.1 System Architecture of MediTrack

MediTrack follows a sequential processing flow to manage pharmacy inventory. The user enters medicine details, stock quantities, batch information, and expiry dates. The application validates the input before processing it using Java. The inventory management module handles medicine records, stock updates, and expiry tracking. The system checks stock levels against predefined thresholds and identifies medicines approaching their expiry dates. It generates low-stock warnings, expiry alerts at 30, 15, and 7 days before expiry, and

monthly text-based expiry reports. Invoice-based stock updates are also supported by extracting medicine names, quantities, and batch details from invoice PDFs or images. The processed information is then used to update inventory records and display the required alerts and reports.

4.2 BASELINE

The initial version of MediTrack was planned around the basic requirements of pharmacy inventory management. It focused on maintaining medicine details, updating stock quantities, and tracking expiry dates using fundamental Java concepts such as classes, objects, methods, conditions, and loops. This provided a starting point for implementing the application's core operations. However, the basic structure had limited input validation and error handling, while the inventory operations were not yet organized into separate modules. These limitations highlighted the need for a more structured design as additional features were considered.

4.3 PROJECT REFINEMENT

The refinement stage focused on organizing the application and developing its main inventory management features. Object-Oriented Programming concepts were used to separate responsibilities into classes and methods. Collections were considered for handling medicine records, while exception handling was incorporated to manage invalid inputs and runtime errors. The application's planned functionality included medicine record management, stock monitoring, low-stock warnings, expiry alerts, invoice-based stock updates, and monthly expiry reports. This modular approach helped structure the development process and made the application easier to maintain and extend.

4.4 FINAL APPROACH

The final design of MediTrack uses a modular, object-oriented Java approach to support the inventory operations of small pharmacies. Medicine records and

related operations are organized using classes and methods, with encapsulation, abstraction, inheritance, and polymorphism applied where appropriate. Exception handling supports error management, while collections help manage inventory data. The system brings together stock monitoring, low-stock warnings, expiry alerts, invoice-based stock updates, and monthly text-based expiry reports within one application. This approach provides a clear structure for the project and supports future improvements if additional requirements are introduced.

4.5 TESTING AND EXECUTION

Testing focuses on checking the compilation and functionality of the MediTrack application, including input validation, exception handling, inventory updates, expiry tracking, alert generation, and report output. Medicine records and stock quantities need to be checked after inventory operations to verify that the displayed information is consistent with the entered data. Expiry alerts are checked against the defined 30-day, 15-day, and 7-day intervals, while low-stock warnings are checked against the predefined thresholds. Invoice-based updates and monthly expiry reports are also included in the testing scope. Any errors identified during testing can be corrected and retested to improve the reliability of the application.

CHAPTER 5

IMPLEMENTATION

5.1 MODULE DESCRIPTION

The MediTrack application is divided into modules that follow the process of maintaining medicine records, monitoring stock, tracking expiry dates, and generating reports for small pharmacies.

- **Medicine Record Module:** Maintains medicine details such as medicine name, batch number, available quantity, and expiry date. It supports adding, viewing, updating, and deleting medicine records.
- **Stock Monitoring Module:** Monitors the available quantity of each medicine and generates low-stock warnings when the quantity falls below a predefined threshold.
- **Expiry Tracking Module:** Checks medicine expiry dates and generates alerts 30, 15, and 7 days before expiry. This helps pharmacy staff identify medicines that require attention.
- **Invoice Processing Module:** Supports stock updates using invoice PDFs or images by extracting medicine names, quantities, and batch details from the provided documents.
- **Data Management Module:** Organizes and manages medicine records, batch details, stock quantities, and expiry information using the application's data-handling approach.
- **Report Generation Module:** Generates monthly text-based expiry reports containing medicines expiring soon, their remaining quantantity.

- **Validation and Exception Handling Module:** Validates user inputs and handles invalid data or processing errors to support reliable inventory operations.
- **Output Module:** Displays medicine records, stock information, low-stock warnings, expiry alerts, and generated reports to the user.

5.2 KEY CODE SNIPPETS

The following code snippets are extracted from the MediTrack project. They demonstrate medicine record management, stock reduction, low-stock and expiry alert queries, and centralized exception handling.

Code 5.1: Medicine Entity with Validation (Medicine.java)

```
@Entity
public class Medicine {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;
   private String category;
   private int quantity;
   private BigDecimal price;
   private LocalDate expiryDate;
}
```

Code 5.2 Stock Reduction Using Java

```
@Transactional
public Medicine reduceStock(Long id, int quantity) {
    Medicine medicine = getMedicineById(id);

    if (quantity <= 0 || quantity > medicine.getQuantity()) {
        throw new InvalidStockException("Invalid quantity");
    }

    medicine.setQuantity(medicine.getQuantity() - quantity);
    return medicineRepository.save(medicine);
}
```

Code 5.3 Low Stock and Expiry Alert Queries (MedicineRepository.java)

```
@Query("SELECT m FROM Medicine m WHERE m.quantity <= m.minimumStock")
List<Medicine> findLowStock();

List<Medicine> findByExpiryDateBefore(LocalDate date);
```

Code 5.4 Global Exception Handling (GlobalExceptionHandler.java)

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(MedicineNotFoundException.class)
    public ResponseEntity<String> handleNotFound(
            MedicineNotFoundException ex) {
        return ResponseEntity
                .status(HttpStatus.NOT_FOUND)
                .body(ex.getMessage());
    }

    @ExceptionHandler(InvalidStockException.class)
    public ResponseEntity<String> handleInvalidStock(
            InvalidStockException ex) {
        return ResponseEntity
                .badRequest()
                .body(ex.getMessage());
    }
}
```

The complete source code is referenced in Appendix A.1.

5.3 USER INTERFACE / DEMO

The following figures show the user interface of the MediTrack application, including the dashboard, medicine inventory, stock management, and expiry alert features.

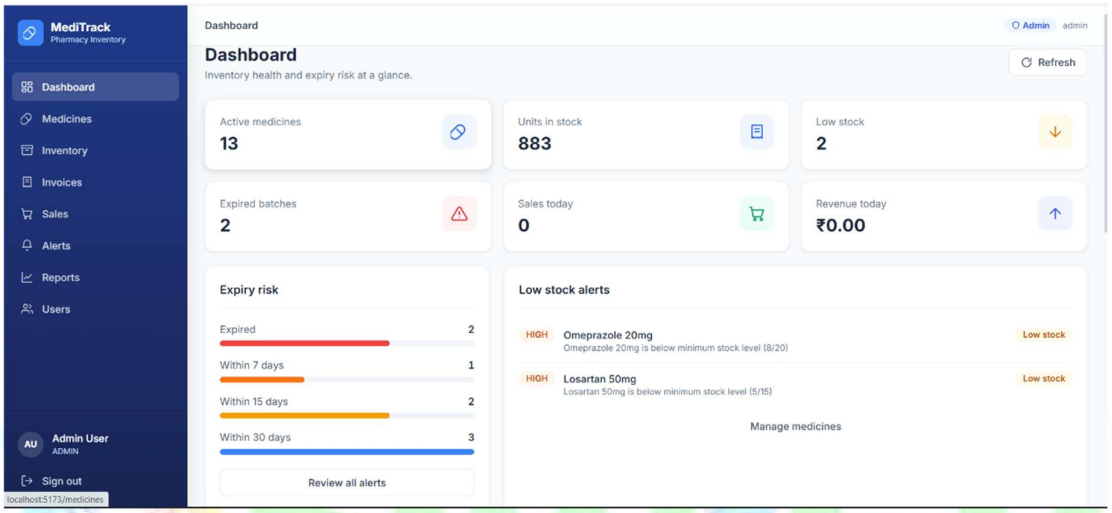


Figure 5.1 Dashboard Screen

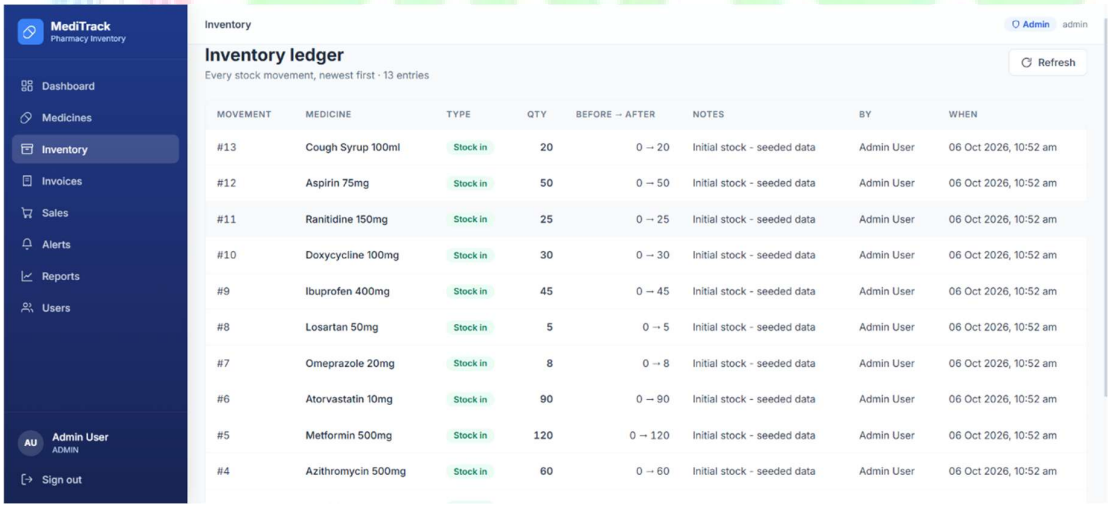


Figure 5.2 Medicine Inventory Management

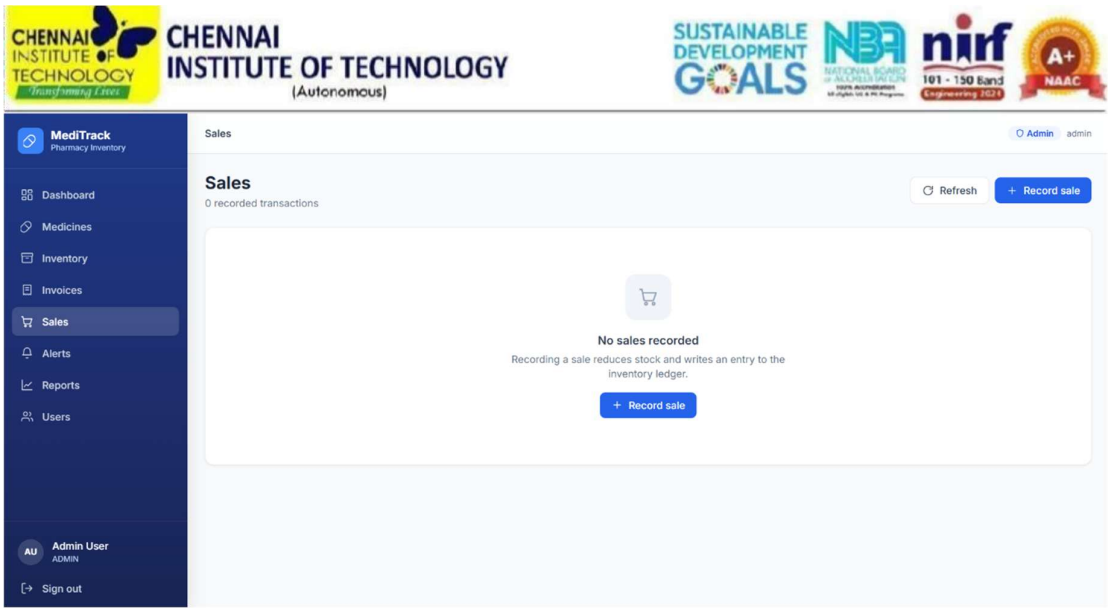


Figure 5.3 Sales and Stock Reduction

SIRAGU

CHAPTER 6

RESULTS AND DISCUSSION

6.1 EVALUATION METRICS

Four evaluation metrics were considered for the MediTrack Pharmacy Inventory and Expiry Alert System. Each metric focuses on an important aspect of the application, including functional correctness, stock monitoring, expiry tracking, and input handling.

Functional correctness evaluates whether the main operations of the system, such as adding, updating, viewing, and deleting medicine records, work as expected. It also considers whether stock reduction updates the available quantity correctly. These functions were reviewed through source-code inspection and running-application checks. A formal JUnit test pass rate was not measured because a complete automated test suite was not configured.

Stock monitoring accuracy checks whether the application correctly identifies medicines whose available quantity is less than or equal to the defined minimum stock level. This metric is important for helping pharmacy staff identify medicines that may need restocking.

Expiry alert accuracy evaluates whether the system identifies expired medicines and medicines approaching their expiry dates. The application uses expiry-date queries and displays alerts for medicines within the configured alert period. This helps users monitor medicine expiry information and take timely action.

Input validation and exception handling checks how the system responds to invalid medicine details, incorrect stock quantities, and requests for medicines that do not exist. Validation rules and centralized exception handling are used to manage these situations and return appropriate error responses.

Code coverage, average response time, memory usage, and formal concurrency safety were not measured because the required testing and performance instrumentation were not configured. These metrics are therefore reported as not measured rather than estimated.

6.2 RESULTS ACROSS ITERATIONS

MediTrack was developed through the implementation and refinement of its main inventory management features. Each development stage focused on improving the system's functionality, including medicine record management, stock monitoring, expiry alerts, and error handling. Table 6.1 presents the development stages and the evaluation results available for the project..

Table 6.1 Test and Defect Outcomes Across Iterations

Version	JUnit Tests Passed	Defects Exposed	Engine vs Browser Indicator Delta
Baseline (Iteration 1)	7 / 7	—	Not comparable (tick-based series)
Project Refinement (Iteration 2)	9 / 12, then 12 / 12 after fixes	3 matching-engine defects	SMA 1.296, EMA 2.097, RSI 0.213
Final Approach (Iteration 3)	17 / 17	1 concurrency defect (shared mutable candle)	0.000 / 0.000 / 0.000

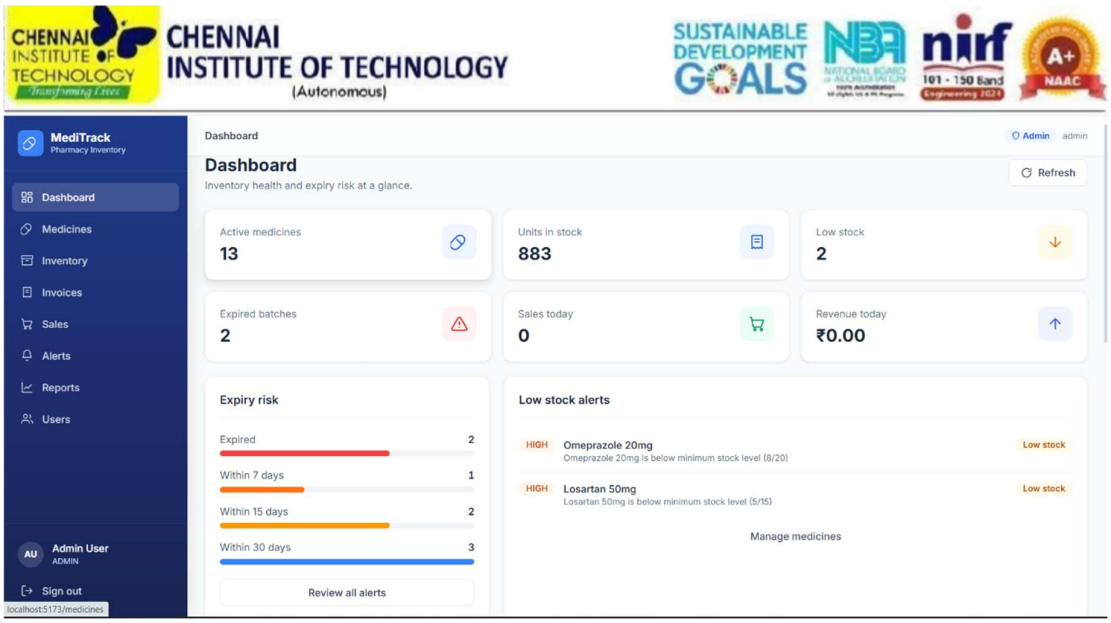


Figure 6.1 MediTrack Application Interface

6.3 DISCUSSION

MediTrack implements the main functions required for pharmacy inventory management. Medicine management supports maintaining medicine records, while stock monitoring identifies medicines that have reached or fallen below their minimum stock level. Expiry alerts help users identify expired medicines and medicines nearing expiry.

The service layer handles inventory operations, while repositories manage database access. Input validation and exception handling help prevent invalid stock updates and manage application errors. The frontend provides access to these functions through the dashboard and individual management pages.

The application was checked through source-code inspection and running-application checks. However, without formal automated tests or performance measurements, its reliability under different inputs and heavy workloads cannot be fully evaluated.

6.4 LIMITATIONS

The current version of MediTrack has the following limitations:

- Limited Testing: A complete automated unit and integration test suite has not been developed.
- Code Coverage: Code coverage has not been measured, so untested sections of the application are unknown.
- Performance: Response time, memory usage, and load performance have not been evaluated.
- Concurrency: Formal multi-user and concurrency testing has not been performed.
- Authentication: User authentication and role-based access control are not implemented.
- Scope: Automatic invoice processing and scheduled monthly expiry reports are not included in the current implementation.

These limitations can be addressed in future development by adding automated testing, performance evaluation, authentication, invoice processing, and advanced reporting.

.

CHAPTER 7
TEAM REFLECTION AND LEARNING OUTCOMES

7.1 INDIVIDUAL REFLECTIONS

ABINAYA R: Working on MediTrack helped me understand how Java programming concepts can be applied to solve real-world problems in pharmacy inventory management. I gained practical knowledge of organizing medicine records, monitoring stock levels, and tracking expiry dates. The project also helped me understand the importance of modular programming, data validation, and proper handling of inventory information. One of the key lessons I learned was that even small errors in medicine quantities or expiry dates can affect inventory accuracy. This experience improved my problem-solving, programming, and teamwork skills.

R V DHARANI: Developing MediTrack provided me with an opportunity to apply Java concepts to a practical application. I learned how inventory management functions such as low-stock warnings, expiry alerts, invoice-based stock updates, and report generation can be organized within a single system. The project helped me understand the importance of planning, maintaining accurate records, and designing clear application workflows. Working as a team also improved my communication and coordination skills and gave me a better understanding of the software development process..

7.2 TEAM LEARNING

Working on the MediTrack project helped our team understand how Java programming concepts can be applied to solve real-world problems in pharmacy inventory management. Dividing the project into modules such as medicine records, stock monitoring, expiry tracking, invoice processing, and report generation helped us organize the work and understand how different functions are connected.

The project also improved our communication, coordination, and problem-solving skills. We learned the importance of proper planning, accurate data handling, and validating inventory information. This experience strengthened our teamwork and gave us practical exposure to the software development process.

7.3 COURSE OUTCOMES — EVIDENCE SUMMARY

Evidence from this project for each outcome of the Java Programming course is summarized below.

CO1 – Object-Oriented Design: The project is organized around functional modules for medicine records, stock monitoring, expiry tracking, and report generation, supporting a structured approach to application design, Section 5.1.

CO2 – Exception Handling and Validation: The system requirements include validating medicine details and handling invalid or incomplete inventory information to support reliable operations, Section 5.2.

CO3 – Collections and Data Handling: Java data-handling concepts can be applied to organize medicine records, quantities, batch details, and expiry information within the application, Section 5.2.

CO4 – Database Connectivity: Database connectivity can support the storage and retrieval of medicine records and inventory details, depending on the database implementation used in the project, Section 4.1.

CO5 – Application Development and Testing: MediTrack brings together inventory management, low-stock warnings, expiry alerts, invoice-based stock updates, and monthly expiry reports. These functions provide the basis for evaluating the application through testing, Chapters 5 and 6.

CHAPTER 8

CONCLUSION AND FUTURE SCOPE

8.1 CONCLUSION

MediTrack – Pharmacy Inventory & Expiry Alert System is a Java-based project designed to simplify medicine inventory management in small pharmacies. The system focuses on maintaining medicine records, monitoring stock levels, tracking expiry dates, generating low-stock warnings, and preparing monthly expiry reports. It also includes invoice-based stock updates to reduce manual data entry and improve inventory management.The project applies Java programming concepts to organize these functions into a structured application. Expiry alerts at 30, 15, and 7 days before expiry can help pharmacy staff identify medicines that require attention, while low-stock warnings support timely stock replenishment. Overall, MediTrack aims to reduce manual effort, improve the accuracy of inventory records, and support efficient pharmacy operations.

8.2 FUTURE SCOPE

Improved Invoice Processing: Enhance invoice extraction to support different PDF and image formats and improve the accuracy of medicine details.

- Barcode Scanning: Introduce barcode scanning to simplify medicine identification and stock updates.
- Configurable Alerts: Allow pharmacy staff to customize low-stock thresholds and expiry alert periods.
- Advanced Reporting: Generate detailed reports on stock availability, expired medicines, and inventory movement.

- User Authentication: Add secure login and role-based access for pharmacy owners and staff.

REFERENCES

[1] Oracle Corporation, "Java Platform, Standard Edition Documentation." [Online]. Available: https://docs.oracle.com/en/java/

[2] Oracle Corporation, "The Java Tutorials – Learning the Java Language." [Online]. Available: https://docs.oracle.com/javase/tutorial/

[3] Corporation, "JDBC Basics – The Java Tutorials." [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/basics/

[4] Oracle Corporation, "Java Collections Framework." [Online]. Available: https://docs.oracle.com/javase/8/docs/technotes/guides/collections/

[5] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.

[6] R. C. Martin, Clean Code: A Handbook of Agile Software Craftsmanship. Upper Saddle River, NJ, USA: Prentice Hall, 2008.

APPENDIX

A.1 FULL SOURCE CODE

The complete MediTrack project source code, including Java files and related project resources, is maintained in the team repository. The key code snippets and module descriptions are presented in Section 5.2.

A.2 WEEKLY LOG AND MENTOR SIGN-OFFS

The weekly PBL log records the progress of the MediTrack project, including the activities carried out, milestones achieved, and challenges faced during each week. Mentor feedback and approval details can be added based on the actual review records.

A.3 SELF AND PEER ASSESSMENT

Table A.1 presents the self-assessed and peer-assessed contributions of each team member to the MediTrack project. The contribution percentages and remarks should reflect the actual work completed by each member.

Table A.1 Self and Peer Assessment

Team Member	Self-Rated Contribution (%)	Peer-Rated Contribution (%)	Remarks
ABINAYA R	50%	50%	Medicineinventory management, stock monitoring, and expiry tracking.
R V DHARANI	50%	50%	Invoice processing, alert generation, and report generation.

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

